Лабораторная работа 1.13. Подключение веб приложения к базе данных

Цель лабораторной работы.

Получить практические навыки в создании подключения веб-приложения к базе данных PostgreSQL и выполнении запросов на вывод информации из таблиц базы данных.

Введение

Для подключения приложения к базе данных необходимо в репозитории Git создать ветку dev5 на основе ветки master. После проверки преподавателем работоспособности веб-приложения ветка dev5 должна быть слита с веткой master.

Создание базы данных

Разрабатываемое веб-приложение должно взаимодействовать с базой данных PostgreSQL. Для работы с веб-приложением создадим базу данных persons с двумя таблицами roles и persons.

Для создания базы данных будем использовать следующий скрипт.

```
CREATE DATABASE persons
```

Скрипты для создания таблиц roles и persons

```
DROP TABLE IF EXISTS role, persons;

CREATE TABLE IF NOT EXISTS persons
(
   id serial NOT NULL ,
   roleid integer NOT NULL,
   firstname character varying(30) ,
   lastname character varying(30) ,
   phone character varying(30) ,
   email character varying(30) ,
   CONSTRAINT persons_pkey PRIMARY KEY (id),
   CONSTRAINT persons_roleid_fkey FOREIGN KEY (roleid)
      REFERENCES public.roles (id)
)

CREATE TABLE IF NOT EXISTS roles
```

```
(
   id serial NOT NULL ,
   rolename character varying(15),
   CONSTRAINT roles_pkey PRIMARY KEY (id)
)
```

Далее необходимо заполнить базу данных тестовыми данными (рис.1, 2).

Data Output | Explain | Messages

	id [PK] integer	rolename character varying (15)
1	1	директор
2	2	бухгалтер
3	3	менеджер

Рисунок 1 – Данные таблицы roles

Data Output | Explain | Messages

	id [PK] integer	roleid integer	firstname character varying (30)	lastname character varying (30)	phone character varying (30)	email character varying (30)
1	1	1	Иван	Иванов	+7(961)-257-32-61	ivanov@mail.ru
2	2	2	Петр	Петров	+7(961)-262-33-25	petrov@mail.ru
3	3	3	Сергей	Сергеев	+7(961)-288-25-00	sergeev@mail.ru

Рисунок 2 – Данные таблицы persons

Создание соединения с базой данных

Для взаимодействия с базой данный необходимо к проекту подключить драйвер PostgreSQL Java. Драйвер postgresql-42.5.4.jar следует скачать из интернета и поместить в папку WEB-INF/lib (рис. 3)

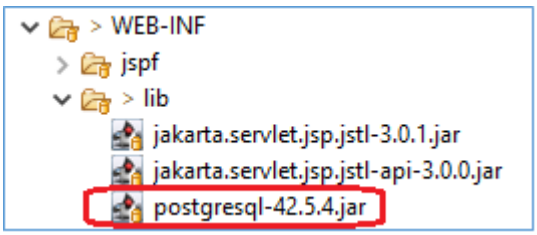


Рисунок 3 – Подключение в проект драйвера PostgreSQL Java

Данные для соединения с базой данных целесообразно подготовить в конфигурационном файле config.proporties в виде «ключ - значение», который следует поместить в папку /persons/src/main/java/config проекта (листинг 1).

Листинг 1

```
db.driver.class=org.postgresql.Driver
db.url=jdbc:postgresql://localhost:5433/persons
db.login=postgres
db.password=********
```

Пароль и логин задаются для созданной базы данных.

Файл config.proporties следует поместить в папку config проекта (рис. 4).

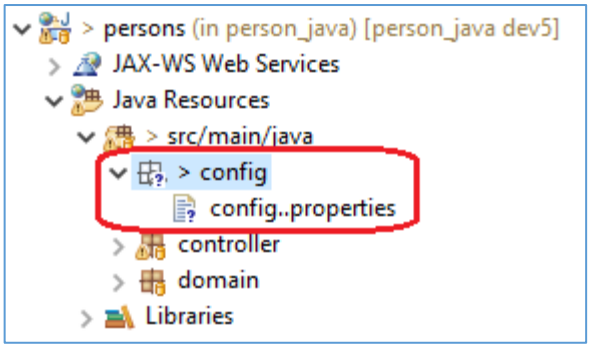


Рисунок 4 – Размещение в проекте файла config.proporties

Сформируем пакет DAO (Data Access Object), который будет содержать компоненты, обеспечивающие соединение с базой данных.

Для получения в приложении свойств, требуемых при подключении к базе данных, создадим класс ConnectionProperty в пакете dao.

Листинг 2.

```
package dao;

import java.io.FileNotFoundException;
import java.io.IOException;
import java.util.Properties;

/**
 * Класс формирования свойств соединения с БД по конфигурационному
файлу
 */
public class ConnectionProperty {
```

```
	// Имя конфигурационного файла
	public static final String CONFIG_NAME = "config.properties";
	// Свойства конфигурации
	public static final Properties PROPERTY_COFIG = new Properties();

	public ConnectionProperty() throws FileNotFoundException,
IOException {
		ClassLoader classLoader = getClass().getClassLoader();

	PROPERTY_COFIG.load(classLoader.getResourceAsStream("config/" +
CONFIG_NAME));
	}

	// Получить значение параметра из конфигурации по имени свойства
	public static String getProperty(String property) {
		return PROPERTY_COFIG.getProperty(property);
	}
}
```

В классе ConnectionProperty поле CONFIG_NAME, определяет имя конфигурационного файла и поле PROPERTY_COFIG – экземпляр свойств. В конструкторе класса осуществляется загрузка свойств из файла.

```
ClassLoader classLoader = getClass().getClassLoader();
PROPERTY_COFIG.load(classLoader.getResourceAsStream("config/"          +
CONFIG_NAME));
```

Метод getProperty( ) позволяет получить конкретное свойство из конфигурационного файла по ключу.

```
public static String getProperty(String property) {
	return PROPERTY_COFIG.getProperty(property);
}
```

Интерфейс ConnectionBuilder определяет метод getConnection( ) получения соединения с базой данных.

Листинг 3.

```
package dao;

import java.sql.Connection;
import java.sql.SQLException;
```

```
// Интерфейс получения соединения с базой данных
public interface ConnectionBuilder {
	Connection getConnection( ) throws SQLException;
}
```

Класс DbConnectionBuilder выполняет функцию построения соединения с базой данных для приложения.

Листинг 4.

```
package dao;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 * Класс Построителя соединения с базой данных
 */
public class DbConnectionBuilder implements ConnectionBuilder {

	public DbConnectionBuilder() {
		try {
			// Настройка драйвера базы данных
	Class.forName(ConnectionProperty.getProperty("db.driver.class"));
		} catch (ClassNotFoundException ex) {
			ex.printStackTrace();
		}
	}

	// Получение соединения с базой данных
	@Override
	public Connection getConnection() throws SQLException {
		String url = ConnectionProperty.getProperty("db.url");
		String login = ConnectionProperty.getProperty("db.login");
		String password =
ConnectionProperty.getProperty("db.password");
		return DriverManager.getConnection(url, login, password);
	}
}
```

В конструкторе класса подключается драйвер PostgreSQL, используя свойство из конфигурационного файла.

```
Class.forName(ConnectionProperty.getProperty("db.driver.class"));
```

Метод getConnection() возвращает экземпляр класса Connection соединения с базой данных.

Репозитории хранения данных

Обобщенный интерфейс RepositoryDAO<T> определяет методы работы веб-приложения с таблицами базы данных (листинг 5).

Листинг 5

```
package dao;

import java.util.List;
import exception.DAOException;

public interface RepositoryDAO<T> {

	// Добавление сущности в таблицу базы данных
	// возвращает ID добавленного должности
	public Long insert(T o) throws DAOException;

	// Редактирование сущности
	public void update(T o) throws DAOException;

	// Удаление сущности
	public void delete(Long Id) throws DAOException;

	// Поиск сущности по Id
	public T findById(Long Id) throws DAOException;

	// Получение списка сущностей
	public List<T> findAll() throws DAOException;
}
```

При реализации интерфейса в качестве типизированного параметра будут применяться классы предметной области Role и Persons/

Класс RoleDbDAO содержит реализацию методов работы веб-приложения с таблицей roles (Должности) базы данных (листинг 6).

Листинг 6.

```
package dao;

import java.sql.Connection;
```

```
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.LinkedList;
import java.util.List;

import domain.Role;
import exception.DAOException;

/**
 * Класс реализации функций взаимодействия с базой данных для таблицы
roles
 * (Должности)
 */

public class RoleDbDAO implements RepositoryDAO<Role> {

	public RoleDbDAO() {
		// TODO Auto-generated constructor stub
	}

	// SQL-запросы к таблице roles базы данных
	private static final String select_all_role = "SELECT id,
rolename FROM roles ORDER BY rolename ASC";
	private static final String select_role_ById = "SELECT id,
rolename FROM roles WHERE id = ?";
	private static final String insert_role = "INSERT INTO
roles(rolename) VALUES(?)";
	private static final String edit_role = "UPDATE roles SET
rolename = ? WHERE id = ?";
	private static final String delete_role = "DELETE FROM roles
WHERE id = ?";

	// Создание соединения с базой данных

	private ConnectionBuilder builder = new DbConnectionBuilder();

	private Connection getConnection() throws SQLException {
		return builder.getConnection();
	}

	// Добавление новой должности
	@Override
	public Long insert (Role role) throws DAOException {
		try (Connection con = getConnection();
				PreparedStatement pst =
con.prepareStatement(insert_role, new String[] { "id" })) {
			Long Id = -1L;
			pst.setString(1, role.getNamerole());
			pst.executeUpdate();
			ResultSet gk = pst.getGeneratedKeys();
```

```
                if (gk.next()) {
                    Id = gk.getLong("id");
                }
                gk.close();
                return Id;
            } catch (Exception e) {
                throw new DAOException(e);
            }
        }

        // Редактирование должности
        @Override
        public void update(Role role) throws DAOException {
            try (Connection con = getConnection(); PreparedStatement pst
= con.prepareStatement(edit_role)) {
                pst.setString(1, role.getNamerole());
                pst.setLong(2, role.getId());

                pst.executeUpdate();
            } catch (Exception e) {
                throw new DAOException(e);
            }
        }

        // Удаление должности
        @Override
        public void delete(Long Id) throws DAOException {
            try (Connection con = getConnection(); PreparedStatement pst
= con.prepareStatement(delete_role)) {
                pst.setLong(1, Id);
                pst.executeUpdate();
            } catch (Exception e) {
                throw new DAOException(e);
            }
        }

        // Поиск должности по Id
        @Override
        public Role findById(Long Id) throws DAOException {
            Role role = null;
            try (Connection con = getConnection()) {
                PreparedStatement pst =
con.prepareStatement(select_role_ById);
                pst.setLong(1, Id);
                ResultSet rs = pst.executeQuery();
                if (rs.next()) {
                    role = fillRole(rs);
                }
                rs.close();
                pst.close();
            } catch (Exception e) {
```

```
                throw new DAOException(e);
            }
            return role;
        }

        // Формирование списка всех должностей
        @Override
        public List<Role> findAll() throws DAOException {
            List<Role> list = new LinkedList<>();
            try (Connection con = getConnection();
                    PreparedStatement pst =
con.prepareStatement(select_all_role);
                    ResultSet rs = pst.executeQuery()) {
                while (rs.next()) {
                    list.add(fillRole(rs));
                }
                rs.close();
            } catch (Exception e) {
                throw new DAOException(e);
            }
            return list;
        }

        // Формирование класса Должности по результатам запроса к БД
        private Role fillRole(ResultSet rs) throws SQLException {
            Role role = new Role();
            role.setId(rs.getLong("id"));
            role.setNamerole(rs.getString("rolename"));
            return role;
        }

        // Поиск должности по id из списка должностей
        public Role FindById(Long id, List<Role> roles) {
            if (roles != null) {
                for (Role r : roles) {
                    if ((r.getId()).equals(id)) {
                        return r;
                    }
                }
            } else {
                return null;
            }
            return null;
        }
}
```

Метод FindById(Long id, List<Role> roles) не входит в интерфейс RepositoryDAO и осуществляет поиск класса role (должности) в коллекции должностей по id.

```
    private Role FindById(Long id, ArrayList<Role> roles) {
     if(roles != null) {
          for(Role r: roles) {
               if((r.getId()).equals(id)) {
                    return r;
               }
          }
     }
     else {
          return null;
     }
     return null;
    }
```

Класс PersonDbDAO содержит реализацию методов работы веб-приложения с таблицей persons (Сотрудники) базы данных (листинг 7).

Листинг 7.

```
package dao;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.LinkedList;
import java.util.List;

import domain.Person;
import exception.DAOException;

/**
 * Класс реализации функций взаимодействия с базой данных для таблицы
persons
 * (Сотрудники)
 */

public class PersonDbDAO implements RepositoryDAO<Person> {

     // SQL-запросы к таблице persons базы данных
     private static final String select_all_person = "SELECT id,
firstName, lastname, "
               + "email, phone, roleid FROM persons ORDER BY lastname
ASC";
```

```
	private static final String select_person_ById = "SELECT id,
roleid, firstname, lastname, phone, email "
			+ "FROM persons WHERE id = ?";
	private static final String insert_person = "INSERT INTO persons(
roleid, firstname, lastname, phone, email)"
			+ "VALUES(?,?,?,?,?)";
	private static final String edit_person = "UPDATE persons SET
roleid = ?, "
			+ "firstname = ?, lastname = ?, phone = ?, email = ?
WHERE id = ?";
	private static final String delete_person = "DELETE FROM persons
WHERE id = ?";

	// Создание соединения с базой данных
	private ConnectionBuilder builder = new DbConnectionBuilder();

	private Connection getConnection() throws SQLException {
		return builder.getConnection();
	}

	RoleDbDAO dao = new RoleDbDAO();

	public PersonDbDAO() {
		// TODO Auto-generated constructor stub
	}

	@Override
	public Long insert (Person person) throws DAOException {
		try (Connection con = getConnection();
				PreparedStatement pst =
con.prepareStatement(insert_person, new String[] { "id" })) {
			Long Id = -1L;
			pst.setString(1, person.getFirstName());
			pst.setString(2, person.getLastName());
			pst.setString(3, person.getRole());
			pst.setString(4, person.getPhone());
			pst.setString(4, person.getEmail());
			pst.executeUpdate();
			ResultSet gk = pst.getGeneratedKeys();
			if (gk.next()) {
				Id = gk.getLong("id");
			}
			gk.close();
			return Id;
		} catch (Exception e) {
			throw new DAOException(e);
		}
	}

	@Override
	public void update(Person person) throws DAOException {
```

```
            try (Connection con = getConnection(); PreparedStatement pst
= con.prepareStatement(edit_person)) {
                  pst.setString(1, person.getFirstName());
                  pst.setString(2, person.getLastName());
                  pst.setString(3, person.getRole());
                  pst.setString(4, person.getPhone());
                  pst.setString(4, person.getEmail());
                  pst.executeUpdate();
            } catch (Exception e) {
                  throw new DAOException(e);
            }
      }

      @Override
      public void delete(Long Id) throws DAOException {
            try (Connection con = getConnection(); PreparedStatement pst
= con.prepareStatement(delete_person)) {
                  pst.setLong(1, Id);
                  pst.executeUpdate();
            } catch (Exception e) {
                  throw new DAOException(e);
            }
      }

      @Override
      public Person findById(Long Id) throws DAOException {
            Person person = null;
            try (Connection con = getConnection()) {
                  PreparedStatement pst =
con.prepareStatement(select_person_ById);
                  pst.setLong(1, Id);
                  ResultSet rs = pst.executeQuery();
                  if (rs.next()) {
                        person = fillPerson(rs);
                  }
                  rs.close();
                  pst.close();
            } catch (Exception e) {
                  throw new DAOException(e);
            }
            return person;
      }

      @Override
      public List<Person> findAll() throws DAOException {
            List<Person> list = new LinkedList<>();
            try (Connection con = getConnection();
                        PreparedStatement pst =
con.prepareStatement(select_all_person);
                        ResultSet rs = pst.executeQuery()) {
                  while (rs.next()) {
```

```
                    list.add(fillPerson(rs));
                }
                rs.close();
            } catch (Exception e) {
                throw new DAOException(e);
            }
            return list;
        }

        private Person fillPerson(ResultSet rs) throws SQLException,
DAOException {
            Long idRole = rs.getLong("roleid");
            Person person = new Person();
            person.setId(rs.getLong("id"));
            person.setFirstName(rs.getString("firstname"));
            person.setLastName(rs.getString("lastname"));
            person.setPhone(rs.getString("phone"));
            person.setEmail(rs.getString("email"));
            person.setIdRole(idRole);
            return person;
        }
}
```

Взаимодействие сервлета RoleServlet с базой данных

Код сервлета RoleServlet приведен на листинге 8.

Листинг 8.

```
package controller;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.FileNotFoundException;
import java.io.IOException;
import java.util.List;

import domain.Role;
import exception.DAOException;
import dao.RoleDbDAO;
import dao.ConnectionProperty;

/**
```

```
 * Servlet implementation class RoleServlet
 */
@WebServlet("/role")
public class RoleServlet extends HttpServlet {
	private static final long serialVersionUID = 1L;

	ConnectionProperty prop;

	/**
	 * @throws IOException
	 * @throws FileNotFoundException
	 * @see HttpServlet#HttpServlet()
	 */
	public RoleServlet() throws FileNotFoundException, IOException {
		super();
		// TODO Auto-generated constructor stub
		prop = new ConnectionProperty();
	}

	/**
	 * @see HttpServlet#doGet(HttpServletRequest request,
HttpServletResponse
	 *	response)
	 */
	protected void doGet(HttpServletRequest request,
HttpServletResponse response)
			throws ServletException, IOException {

		response.setContentType("text/html");
		String userPath;
		List<Role> roles;
		RoleDbDAO dao = new RoleDbDAO();

		try {
			roles = dao.findAll();
			request.setAttribute("roles", roles);
		} catch (DAOException e) {
			// TODO Auto-generated catch block
			e.printStackTrace();
		}
		userPath = request.getServletPath();
		if ("/role".equals(userPath)) {

	request.getRequestDispatcher("/views/role.jsp").forward(request,
response);
		}
	}

	/**
	 * @see HttpServlet#doPost(HttpServletRequest request,
HttpServletResponse
```

```
	 *	response)
	 */
	protected void doPost(HttpServletRequest request,
HttpServletResponse response)
			throws ServletException, IOException {
		// TODO Auto-generated method stub
		doGet(request, response);
	}
}
```

Сервлет RoleServlet передает данные с именем roles на jsp-страницу. HTML-разметка страницы role.jsp приведена на листинге 9.

Листинг 9.

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="jakarta.tags.core"%>
<%@ page import="domain.Role"%>

<!DOCTYPE html>
<html>
<meta http-equiv="Content-Type" content="text/html" charset="UTF-8">
<title>Должности</title>
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Roles</title>
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<!-- Bootstrap CSS -->
<link rel="stylesheet" href="css/bootstrap.min.css">
<!-- jQuery -->
<script defer src="js/jquery-3.6.4.js"></script>
<!-- Bootstrap JS + Popper JS -->
<script defer src="js/bootstrap.min.js"></script>
</head>
<body>
	<div class="container-fluid">
		<jsp:include page="/views/header.jsp" />
		<div class="container-fluid">
			<div class="row justify-content-start ">
				<div class="col-8 border bg-light px-4">
					<h3>Список должностей</h3>
					<table class="table">
						<thead>
							<th scope="col">Код</th>
							<th scope="col">Должность</th>
							<th
scope="col">Редактировать</th>
```

```
                                    <th scope="col">Удалить</th>
                                </thead>
                                <tbody>
                                    <c:forEach var="role"
items="${roles}">
                                        <tr>

      <td>${role.getId()}</td>

      <td>${role.getNamerole()}</td>
                                            <td width="20"><a
href="#" role="button"
                                                class="btn btn-
outline-primary"> <img alt="Редактировать"

      src="images/icon-edit.png"></a></td>
                                            <td width="20"><a
href="#" role="button"
                                                class="btn btn-
outline-primary"> <img alt="Удалить"

      src="images/icon-delete.png"></a></td>
                                        </tr>
                                    </c:forEach>
                                </tbody>
                            </table>
                        </div>
                        <div class="col-4 border px-4">
                            <form method="POST" action="">
                                <h3>Новая должность</h3>
                                <div class="mb-3">
                                    <br> <label for="inputRole"
class="col-sm-3 col-form-label">Должность</label>
                                    <div class="col-sm-6">
                                        <input type="text"
name="inputRole" class="form-control"
                                            id="personRole" />
                                    </div>
                                </div>
                                <p>
                                    <br> <br> <br>
                                    <button type="submit" class="btn
btn-primary">Добавить</button>
                                    <br>
                                </p>
                            </form>
                        </div>
                    </div>
                </div>
                <jsp:include page="/views/footer.jsp" />
            </div>
```

```
</body>
</html>
```

После запуска веб-приложения и перехода на страницу Должности в браузере должно быть отображено следующее содержание (рис. 5).

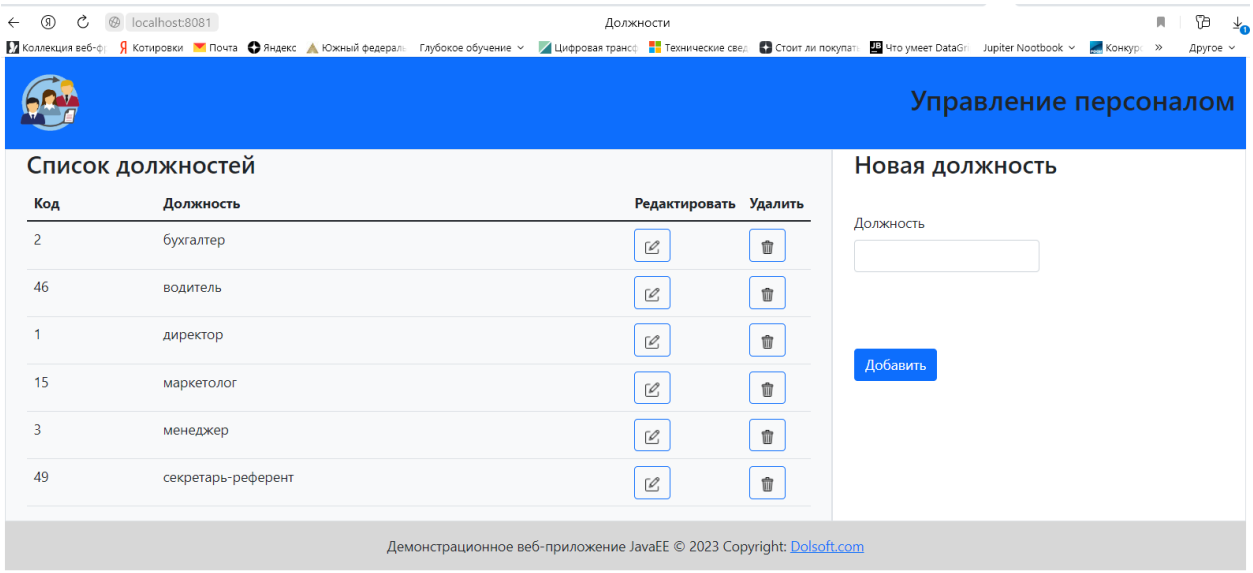


Рисунок 5 – Страница Должности

Взаимодействие сервлета PersonsServlet с базой данных

Код сервлета PersonsServlet приведен на листинге 10.

Листинг 10.

```
package controller;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.FileNotFoundException;
import java.io.IOException;
import java.util.List;

import dao.ConnectionProperty;
import dao.PersonDbDAO;
import dao.RoleDbDAO;

import domain.Person;
import domain.Role;
```

```
import exception.DAOException;

/**
 * Servlet implementation class PersonServlet
 */
@WebServlet("/person")
public class PersonServlet extends HttpServlet {

      private static final long serialVersionUID = 1L;
      ConnectionProperty prop;

    public PersonServlet() throws FileNotFoundException, IOException {
        super();
         prop = new ConnectionProperty();
    }

      protected void doGet(HttpServletRequest request,
HttpServletResponse response)
                throws ServletException, IOException {

            response.setContentType("text/html");
            String userPath;
            List<Person> persons;
            List<Role> roles;

            RoleDbDAO daoRole = new RoleDbDAO();
            PersonDbDAO dao = new PersonDbDAO();

            try {
                  persons = dao.findAll();
                  roles = daoRole.findAll();
                  for (Person person: persons) {

      person.setRole(daoRole.FindById(person.getIdRole(), roles));
                  }

                  request.setAttribute("persons", persons);

            } catch (DAOException e) {
                  // TODO Auto-generated catch block
                  e.printStackTrace();
            }

         userPath = request.getServletPath();
            if("/person".equals(userPath)){

      request.getRequestDispatcher("/views/person.jsp").forward(request
, response);
            }
      }
```

```
	/**
	 * @see HttpServlet#doPost(HttpServletRequest request,
HttpServletResponse response)
	 */
	protected void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException {
		// TODO Auto-generated method stub
		doGet(request, response);
	}
}
```

В сервлете PersonsServlet загружаются данные по должностям и сотрудникам и коллекции roles и persons передаются в представление person.jsp.

Метод FindById(Long id, List<Role> roles) осуществляет поиск класса role (должности) в коллекции должностей по id. Данный метод используется при формировании коллекции persons.

```
while (rs.next()) {
	id = rs.getLong("roleid");
	persons.add(new
		Person(rs.getLong("id"),
		rs.getString("firstName"),
		rs.getString("lastName"),
		rs.getString("email"),
		rs.getString("phone"),
		id,
		FindById(id, roles)
	));
}
```

Сервлет PersonsServlet передает данные с именами roles и persons на jsp-страницу. HTML-разметка страницы persons.jsp приведена на листинге 11.

Листинг 11.

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
	pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="jakarta.tags.core"%>

<%@ page import="domain.Role"%>
<%@ page import="domain.Person"%>


<!DOCTYPE html>
```

```
<html>
<meta http-equiv="Content-Type" content="text/html" charset="UTF-8">
<title>Сотрудники</title>
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Persons</title>
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<!-- Bootstrap CSS -->
<link rel="stylesheet" href="css/bootstrap.min.css">
<!-- jQuery -->
<script defer src="js/jquery.min.js"></script>
<!-- Bootstrap JS + Popper JS -->
<script defer src="js/bootstrap.min.js"></script>

</head>
<body>
<body>
     <div class="container-fluid">
          <jsp:include page="/views/header.jsp" />
          <div class="container-fluid">
          <div class="row justify-content-start ">
          <div class="col-8 border bg-light px-4">
          <h3>Список сотрудников</h3>
          <table class="table">
          <thead>
          <th scope="col">Код</th>
          <th scope="col">Фамилия</th>
          <th scope="col">Имя</th>
          <th scope="col">Должность</th>
          <th scope="col">Телефон</th>
          <th scope="col">Эл.почта</th>
          <th scope="col"></th>
          <th scope="col"></th>
          </thead>
          <tbody>
          <c:forEach var="person" items="${persons}">
          <tr>
          <td>${person.getId()}</td>
          <td>${person.getLastName()}</td>
          <td>${person.getFirstName()}</td>
          <td>${person.getRole()}</td>
          <td>${person.getPhone()}</td>
          <td>${person.getEmail()}</td>
          <td width="20"><a href="#" role="button"
          class="btn btn-outline-primary"> <img alt="Редактировать"
          src="images/icon-edit.png"></a></td>
          <td width="20"><a href="#" role="button"
          class="btn btn-outline-primary"> <img alt="Удалить"
          src="images/icon-delete.png"></a></td>
          </tr>
```

```
</c:forEach>
</tbody>
</table>
</div>
<div class="col-4 border px-4">
<form method="POST" action="">
<h3>Новый сотрудник</h3>
<br>
<div class="mb-3 row">
<label for="lastname"
     class="col-sm-3 col-form-label">Фамилия</label>
<div class="col-sm-7">
<input type="text" class="form-control"
     id="staticLastname" name="lastname" />
</div>
</div>
<div class="mb-3 row">
<label for="firstname"
     class="col-sm-3 col-form-label">Имя</label>
<div class="col-sm-7">
<input type="text" class="form-control"
     id="staticFirstname" name="firstname" />
</div>
</div>
<div class="mb-3 row">
<label for="rolename"
     class="col-sm-3 col-form-label">Должность</label>
<div class="col-sm-7">
<select name="role" class="form-control">
<option>Выберите должность</option>
<c:forEach var="role" items="${roles}">
<option value="${role}">
<c:out value="${role.getNamerole()}"></c:out>
</option>
</c:forEach>
</select>
</div>
</div>
<div class="mb-3 row">
<label for="phone"
     class="col-sm-3 col-form-label">Телефон</label>
<div class="col-sm-7">
<input type="text" class="form-control"
     id="staticphone" name="phone" />
</div>
</div>
<div class="mb-3 row">
<label for="email" class="col-sm-3 col-form-label">Эл.почта
</label>
<div class="col-sm-7">
<input type="text" class="form-control" id="staticemail"
```

```
                name="email" />
            </div>
            </div>
            <p>
            <br>
            <button type="submit"
                class="btn btn-primary">Добавить</button>
            </p>
            </form>
            </div>
            </div>
            </div>
            <jsp:include page="/views/footer.jsp" />
        </div>
</body>
</html>
```

После запуска веб-приложения и перехода на страницу Сотрудники в браузере должно быть отображено следующее содержание (рис. 6).

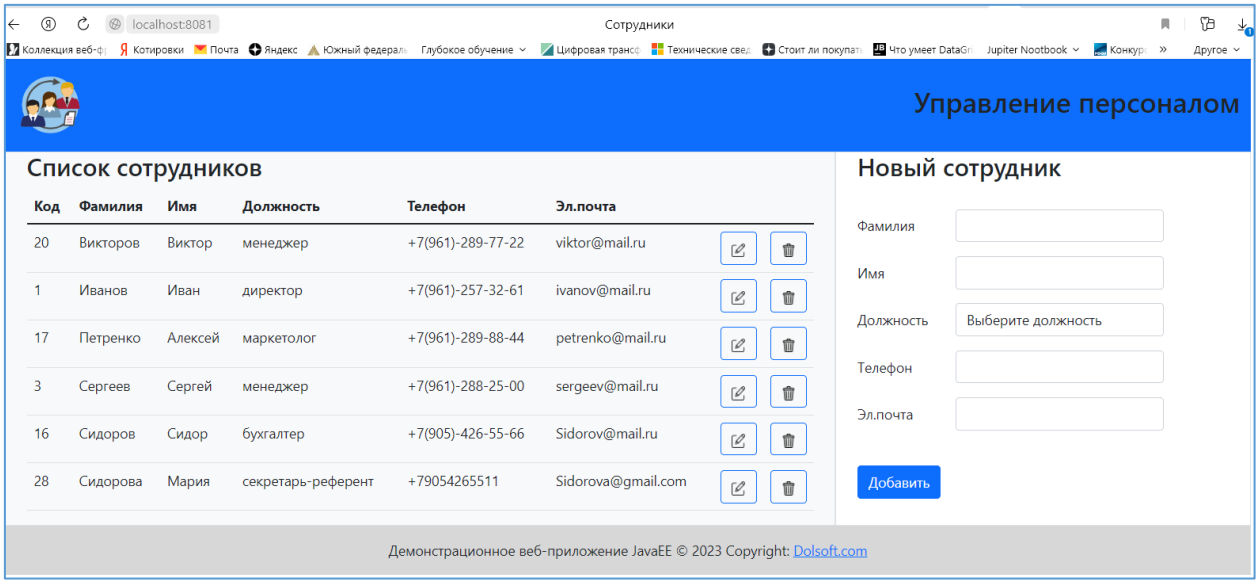


Рисунок 6 – Страница Сотрудники

Задание на лабораторную работу

1. Изучить теоретический материал лекции 1.6 – 1.8.
2. По варианту, определенному преподавателем, разработать пакет DAO и взаимодействие сервлетов с базой данных при выводе всей информации из таблиц базы данных.

3. Продемонстрировать результаты работы веб-приложения предателю.
4. Слить ветку разработки dev5 с основной веткой master.
5. Подготовить отчет по лабораторной работе и загрузить его на учебный портал.